StaffSync – Project Documentation

Full Stack Development with MERN

1. Introduction

- **Project Title:** StaffSync – HR Management Web Application
- **Team Members:** [Add your name and roles here, e.g. Full Stack Developer]

2. Project Overview

Purpose: StaffSync is a web application that helps an organisation manage its HR work in one place. It replaces manual records and scattered messages with a single system for employee data, leave, attendance and announcements.

Features:

- Role-based access for three roles: HR, Manager and Employee
- HR: manage employee data, approve leaves, track attendance, post announcements and news
- Manager: oversee leave requests, manage training and development programs
- Employee: apply for leave, maintain personal records, mark attendance, view announcements
- Secure login using JWT

3. Architecture

Frontend: Built with React, using HTML, CSS, JavaScript and Bootstrap for the interface. React components call the backend through async JavaScript (fetch/axios) and show different screens based on the logged-in role.

Backend: Built with Node.js and Express.js. It exposes REST APIs, checks the JWT on protected routes and applies role checks (HR, Manager, Employee).

Database: MongoDB Atlas (cloud). Data is stored as collections accessed from Node through Mongoose/MongoDB driver. [Add your collections here, e.g. Users, Leaves, Attendance, Announcements, Trainings, with their main fields.]

4. Setup Instructions

Prerequisites:

- Node.js and npm
- Git
- A MongoDB Atlas account (or local MongoDB)

Installation:

1. Clone the repository: `git clone https://github.com/<your-username>/staffsync-hrm.git`
2. Go to the project folder: `cd staffsync-hrm`
3. Install server dependencies: `cd server && npm install`
4. Install client dependencies: `cd ../client && npm install`
5. Create a `.env` file in the server folder with your MongoDB Atlas connection string, JWT secret and port. [Use the exact variable names from your project.]

5. Folder Structure

Client: [Describe your React folder, e.g. components, pages, services/API calls, routes.]

Server: [Describe your Node folder, e.g. models, routes, controllers, middleware, config.]

6. Running the Application

- **Backend:** run `npm start` in the server directory.
- **Frontend:** run `npm start` in the client directory.

Then open the frontend in the browser (usually `http://localhost:3000`). [Change if your scripts or ports differ.]

7. API Documentation

[List every endpoint of your backend in this form.]

Method	Endpoint	Purpose	Access
POST	[/api/auth/login]	Log in and receive a JWT	All

Method	Endpoint	Purpose	Access
[GET]	[/api/...]	[Description]	[Role]

For each important endpoint, add the parameters and one example response.

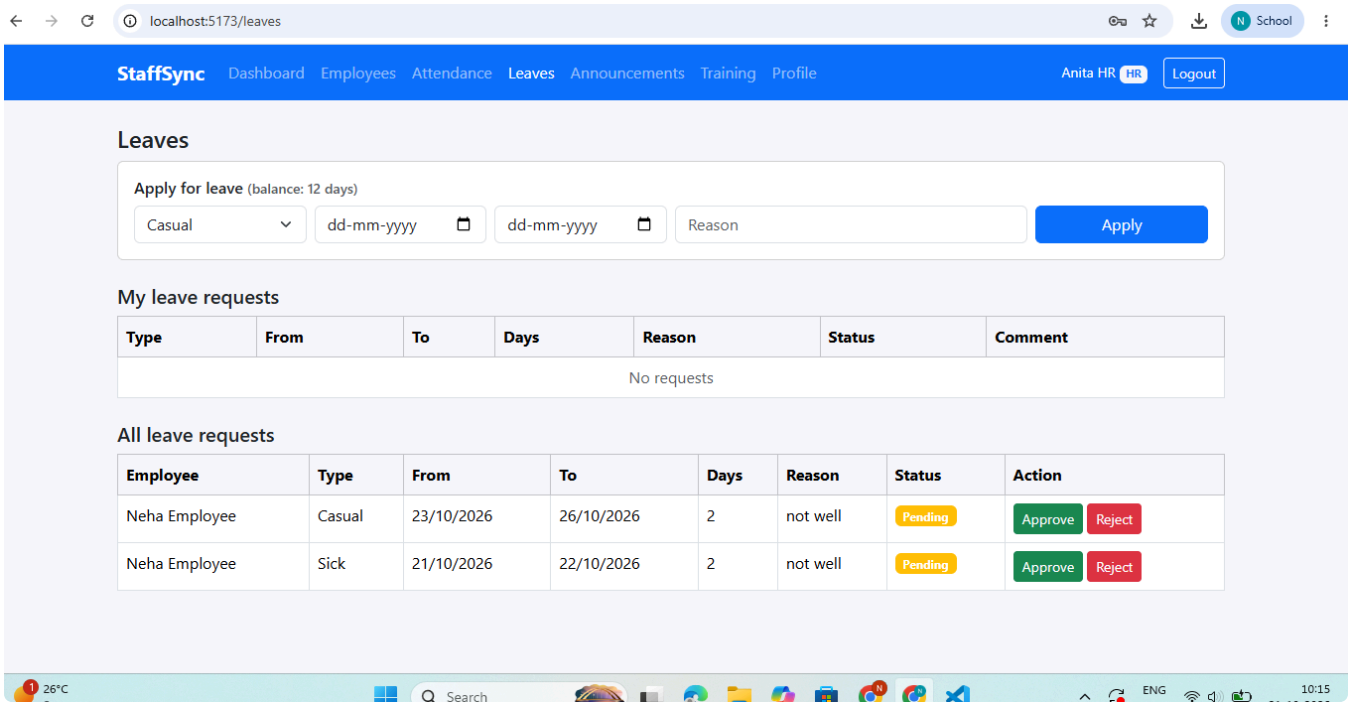
8. Authentication

StaffSync uses JWT (JSON Web Token) based authentication. After a successful login the server returns a signed token. The client sends this token with every request to a protected route, and the server verifies it in middleware. The user's role (HR, Manager or Employee) decides which routes and screens are allowed.

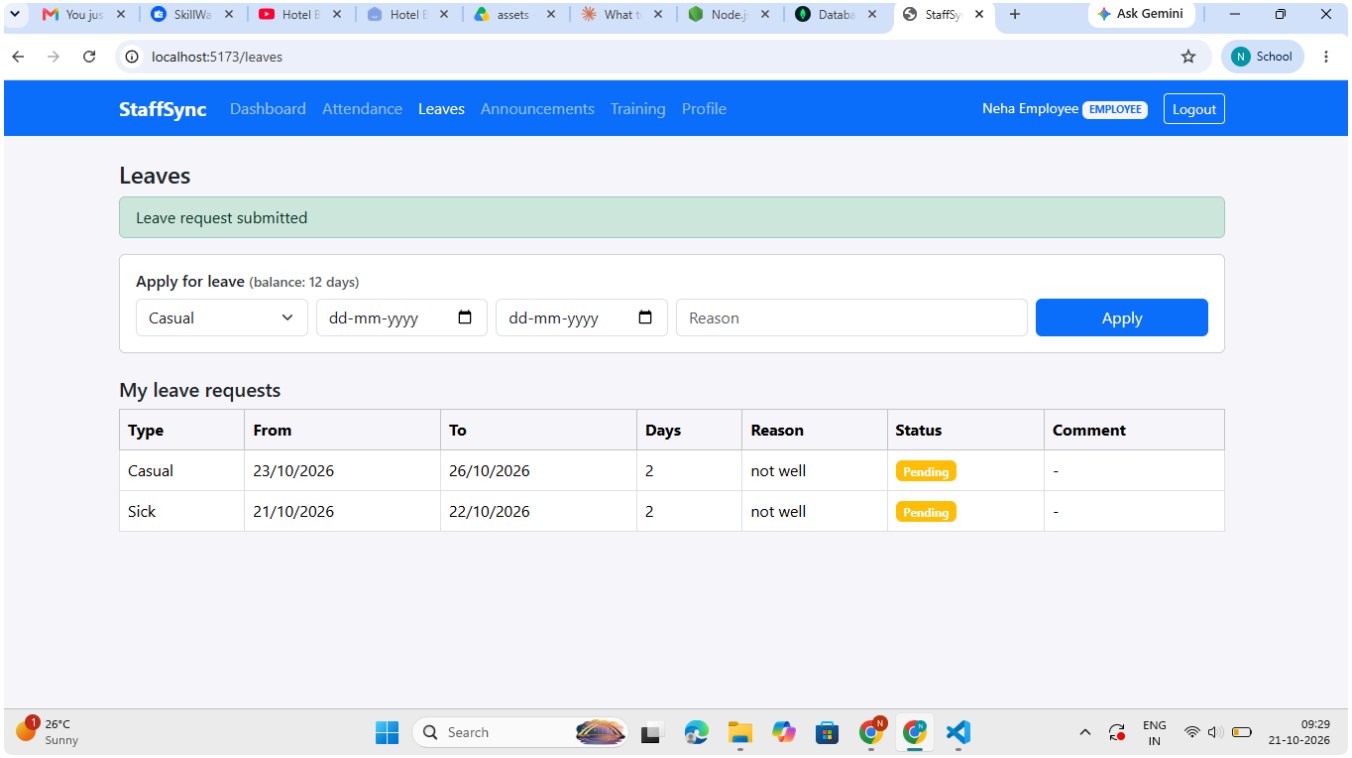
9. User Interface

The Leaves page is shown below for the HR and Employee roles. [Add screenshots of the other screens here: Login, Dashboard, Attendance, Announcements.]

HR view: HR sees every employee's leave request and can approve or reject it.

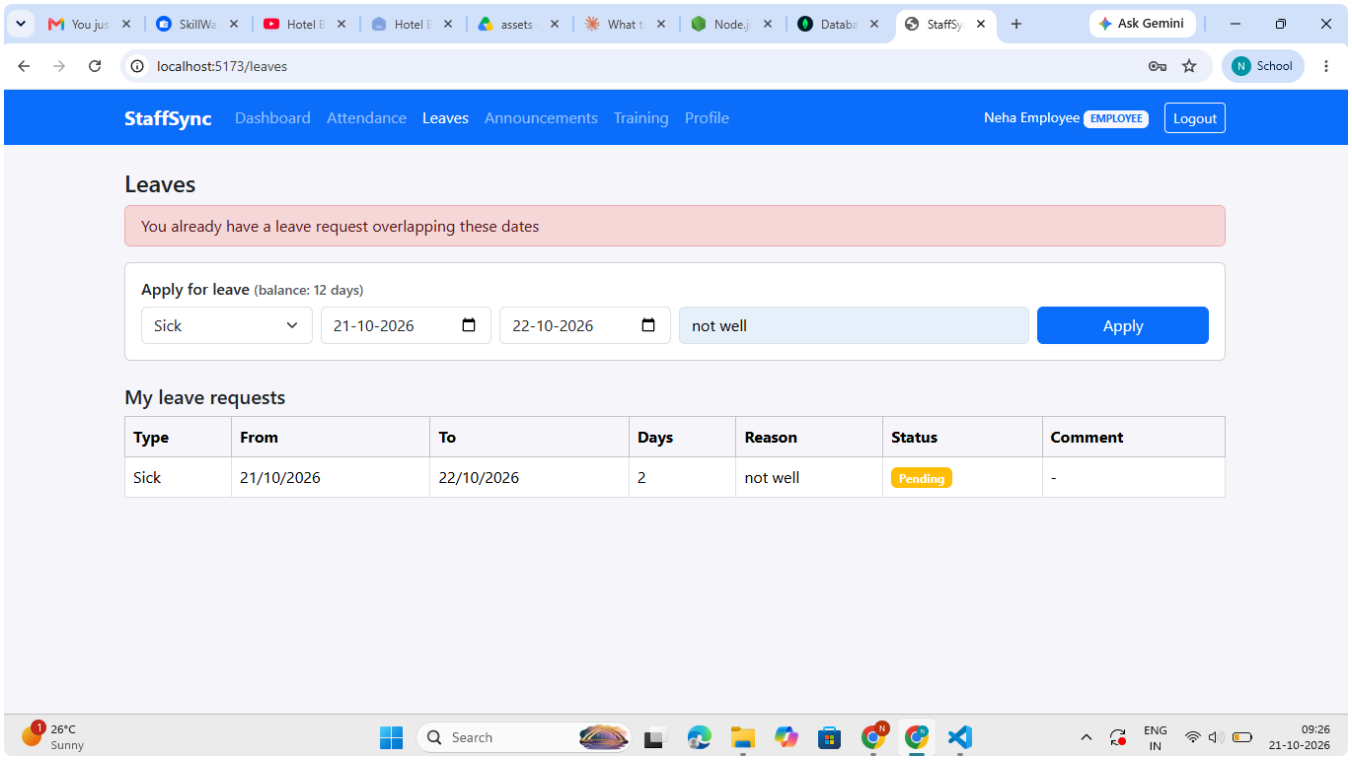


Employee view: the employee applies for leave (balance: 12 days) and sees the status of their requests.



10. Testing

The leave form was tested manually. Applying for dates that overlap an existing request is rejected with the message "You already have a leave request overlapping these dates".



[Describe the rest of your testing here, e.g. each role's flow, API testing with Postman, and tools used.]

11. Screenshots or Demo

See Section 9 for application screenshots. [Add a live demo link here after deployment.] GitHub repository: staffsync-hrm

12. Known Issues

[Write any bugs or limitations you know of. If none, write: No major known issues at present.]

13. Future Enhancements

- Deploy the application (MongoDB Atlas with Render/Vercel)
- Email or in-app notifications for leave approvals
- Payroll and salary slip module
- Reports and charts for attendance and leave